MASTER OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE SYSTEMS

Intelligent Reasoning Systems

Project Report

StockEvent Lens

Group 21

Team members

No.	Full Name	Matriculation ID
1	Lim Hao	A0354214M
2	Pang Claire Fuiyi	A0357738L
3	Polthai Nakhin	A0355086X
4	Le Duc Nghia	A0356261E
5	Kittichaikoonkij Nathan	A0352270M

Contents

1. EXECUTIVE SUMMARY 3
2. PRODUCT DESCRIPTION 4
Project goals 4
3. MARKET RESEARCH 5
3.1 Target Customers 5
3.2 Market Analysis 5
3.3 Market gap 5
4. LITERATURE REVIEW 6
5. System Design 7
5.1 SEC filings 7
5.2 Data Ingestion & Normalisation 7
5.3 Database Storage 7
5.4 Event-Grouping 8
5.5 Event Summarization & Conflict Resolution Architecture 9
5.6 Project Scope & Limitations 10
6. System Development & Implementation 11
6.1 Similarity pipeline 12
6.2 App Screenshots 15
7. Experiments 17
7.1 Objectives and research questions 17
7.2 Labeled synthetic data activity 17
7.3 Experimental procedure 18
7.4 Metrics 18
8. Findings and discussion 19
8.1 LLM performance on similarity comparison (RQ1) 19
8.2 Where the errors come from (RQ2) 20
8.3 What the threshold-plus-LLM design is most likely to miss (RQ3) 21
8.4 Discussion and limitations 22
APPENDICES 24
Appendix of report: Mapped System Functionalities against knowledge, techniques and skills of modular courses: MR, RS, CGS 24
Appendix of report: Actual system screenshots 25
Appendix of report: Installation and User Guide 29
Appendix of report: Project Proposal 33
AI Tool Declaration 34
References 35

List of Figures

Figure 1. Stories in the developer database 9
Figure 2. System architecture diagram. 11
Figure 3. Stock homepage with consolidated events and AI summaries. 15
Figure 4. Event detail view with summary, timeline, original sources and supporting evidence 16
Figure 5. Consolidation and summarization on Batch 1 (test_001–test_005) 17
Figure 6. Label accuracy, macro-F1 and grouping F1 by configuration 19
Figure A1. The news summaries page: each story has an AI summary and links to its original sources 25
Figure A2. Developer mode, batch 1: the first article opens a new story (E1) 25
Figure A3. The second article is merged into E1 as a repeat; the summary is not rewritten 26
Figure A4. The update is merged into E1 with its new fact; the summary becomes version 2 26
Figure A5. The news feed: one story with three sources 27
Figure A6. The conflict: both quotes shown, the summary held for review 27
Figure A7. The news feed marks the story "Conflicting reports — under review" 28
Figure A8. A different NVIDIA event becomes its own story (E2) 28

List of Tables

Table 1. Relationship labels output by Claude 13
Table 2. Configurations compared 18
Table 3. Consolidation results on the 100 scored articles (best LLM value per row in green) 19
Table 4. Gemini 3.8 Flash confusion matrix (rows: expected; columns: predicted) 19
Table 5. Claude Haiku 5.5 confusion matrix (rows: expected; columns: predicted) 20
Table 6. Where each error originates, with its code location (numbers are test article IDs) 20
Table 7. Traceable evidence for each blind spot (batch, event group, test files and quoted text) 21
Table A1. System functionalities mapped to MR, RS and CGS 24

1. EXECUTIVE SUMMARY

A single company event, such as an earnings release, a product launch or a lawsuit, is reported by many outlets within hours. Some reports repeat each other, some add new facts, and some contradict each other. An investor who follows a company has to read all of them to find out what actually happened.

StockEvent Lens groups this coverage into events. It collects SEC EDGAR 8-K filings and Alpaca financial news in real time, and finds earlier coverage of the same company with embedding and full-text search. Claude then decides whether each new report repeats, updates, contradicts or differs from an existing event. Every decision must quote the source text verbatim, or it is downgraded to uncertain and never merged. Each event gets a short summary in which every statement cites its source passages; conflicting reports go to a person for review instead of being resolved by the model. In our configuration, Claude Sonnet 4.5 compares reports and Claude Opus 5 writes the summaries.

On a labelled test set of 125 articles, Claude Sonnet 5.5 groups articles with a pairwise F1 of 0.925 and labels them with a macro F1 of 0.869, without a single false merge. A local 7B model and a rule baseline label far less accurately, with macro F1 of 0.453 and 0.355 (Sections 7 and 8).

2. PRODUCT DESCRIPTION

StockEvent Lens turns a stream of news articles and SEC filings into one continuously updated record per company event, so an investor reads each development once instead of several times.

A single corporate development, such as an acquisition, typically produces a breaking news headline, an SEC Form 8-K filing, follow-up articles, and revised versions of earlier stories. Existing feeds list each of these as a separate item, leaving the reader to work out whether each one is new or a repeat.

StockEvent Lens builds on existing infrastructure rather than replacing it. It uses Benzinga news, delivered through the Alpaca News API, as a secondary source, and company 8-K filings from SEC EDGAR as the primary source. It groups documents about the same development into one event, labels how each new document relates to it (a repeat, an update or a conflict), and flags conflicting reports for a person to review.

Project goals

- Consolidate news items and 8-K filings about the same development into one event record.
- Link each event to the company's 8-K filings and show how many sources support it.
- Decide automatically whether each incoming document creates a new event, updates an existing one, or confirms or contradicts it.
- Explain every event with a summary and facts linked to their source documents.
- Send alerts once per new event or material update, not once per article.
- Evaluate all of the above against baselines that represent how current feed products work.

3. MARKET RESEARCH

3.1 Target Customers

Our target customer is the self-directed investor who actively trades or holds a small portfolio of 10 to 15 large-cap US stocks and reacts to company news. This segment is underserved: institutional tools target professional desks, while consumer finance apps show headlines without grouping them by event. The MVP targets this group first, with a scope small enough for a student project.

3.2 Market Analysis

Our competitors fall into two groups. Retail-facing and institutional platforms both address scattered stock news, but they differ in who they serve and how they present information.

1) Retail-facing platforms

Benzinga Pro. Benzinga Pro's newsfeed combines its wire, press releases, SEC filings, and analyst ratings, filterable by source and watchlist. Public documentation describes a filtered item stream with no event-level grouping. As Alpaca's news supplier, Benzinga competes on workflow rather than content.

Koyfin. Koyfin's Watchlist News combines news, filings, and transcripts in one workspace, filterable by news source, SEC filing type (including 8-Ks), and topic code. Its help pages describe filtered document lists, with no event-level deduplication or merging.

2) Institutional platforms

Bloomberg Terminal. Bloomberg's Key Themes feature, based on the Key News Themes (NSTM) system described in a 2020 ACL paper, clusters related stories and summarizes them into themes. It is delivered on the institutional Bloomberg Terminal, outside the price point and usage pattern of self-directed investors.

RavenPack. RavenPack detects news events automatically and delivers structured event data and sentiment scores through real-time feeds, primarily to quantitative and trading clients. It shows that event detection at scale is established, but its output is machine-readable data rather than an event view for individual investors.

3.3 Market gap

None of the products reviewed presents a single record per development, with labelled updates and conflicts and summaries that quote their sources. That gap, between aggregated news and retail-accessible event tracking, is the space StockEvent Lens aims to occupy.

4. LITERATURE REVIEW

Grouping news by real-world events is an established research task. Fiscus et al. (1999) described the NIST Topic Detection and Tracking (TDT) evaluation, which organised streams of news stories by topic and scored systems by their miss and false-alarm rates. Escoter et al. (2017) grouped business news stories according to the salience of the named entities they mentioned. Bambrick et al. (2020) reported that Bloomberg's NSTM system clustered related stories, removed duplicates, and summarised each cluster into a theme for terminal users. These studies showed that event grouping worked at scale. However, they targeted professional users, and their outputs were topic clusters or summaries rather than a record that checked each news report against the company's own disclosure.

Large language models (LLMs) offered a way to make finer judgements, but the literature cautioned that their reliability had to be measured. Zheng et al. (2023) reported that GPT-4 agreed with human preferences more than 80 percent of the time, matching the agreement between humans, yet they also found that LLM judges favoured answers in the first position and longer answers. For summaries, Laban et al. (2022) found that natural language inference (NLI) models detected inconsistencies effectively once documents were split into sentences. Xu et al. (2024) later showed that zero-shot LLMs, guided by a taxonomy of error types, outperformed specialised non-LLM detectors across ten datasets. Together, these studies suggested that LLM outputs should be checked against independent scorers and simple baselines rather than trusted directly.

A remaining difficulty was deciding when two reports genuinely disagree. De Marneffe et al. (2008) argued that two texts can contradict each other only when they refer to the same event, and they identified numeric mismatch as one common type of contradiction. This project follows that definition: the LLM judge labels a report as conflicting only when it describes the same event and gives an incompatible value for the same metric, unit, and period, and missing information is never treated as a contradiction. Gao et al. (2023) found that even the best LLMs lacked complete citation support about half the time on one of their datasets, which suggested that model citations should be checked rather than trusted. Accordingly, the system verifies that every evidence quote appears verbatim in its cited source, downgrades same-event judgements it cannot verify to "uncertain," and holds summaries for human review whenever a conflict is found.

5. System Design

The system is an automated pipeline that links high-frequency financial news with companies' official 8-K filings. It has three parts: ingestion from several sources, hybrid candidate retrieval, and a cascading decision step in which cheap checks run before any LLM call, which keeps computational cost down. Each event's status changes as articles and filings accumulate, and conflicting reports are held for human review instead of being resolved by the system.

5.1 SEC filings

The U.S. Securities and Exchange Commission is an enforcement agency targeting market manipulation, requiring publicly-listed companies to periodically file financial statements and disclosures. These provide investors with financial insights into the companies, helping them make informed decisions in the financial markets. The companies are required to file different forms for various disclosures, of which our system specifically takes in the 8-K, a necessary form through which unscheduled major events are declared. Filings are collected by the SEC on its public Electronic Data Gathering, Analysis, and Retrieval (EDGAR) database, from which our system will retrieve as well.

5.2 Data Ingestion & Normalisation

The data ingestion layer of the system acquires secondary financial news from the Alpaca API as a supplementary of information to primary SEC EDGAR 8-K filings which are polled every 60 seconds. After ingestion, the data from both sources are transformed into a standardised internal representation to account for their different formats: SEC EDGAR filings are fetched as HTML documents, which are parsed and from which clean text is extracted, while data is received from the Alpaca API as raw JSON objects, whose contents undergo HTML parsing, decoding and normalisation. To prevent downstream corruption of event aggregation, duplicate management targets only source identity (SEC EDGAR & Alpaca) rather than semantic similarity, which is done by indexing documents according to their source identity keys. In the case where an identical document is ingested, the system skips it and does no further work. If the payload for the same source identity has changed, such as an Alpaca article with a newer update timestamp, it is stored as a new revision of the same document rather than as a new document; a payload with an older timestamp is treated as stale and skipped. SEC filings are skipped once completed, and an amended filing (8-K/A) has its own accession number and is therefore its own record.

5.3 Database Storage

Records are stored in PostgreSQL, with the pgvector extension holding the text embeddings. Each source document is stored once, with each of its versions kept as a revision, together with its company links and, for filings, the SEC details. The text of each revision is split into text units, and each unit has an embedding stored under an embedding profile that records the model and chunking settings used. The grouping stage adds its own tables: the analysis status of each revision, the events, the comparisons between a new segment and a candidate event, and the memberships that link a revision to an event together with the units supporting the link. The summary stage adds the published summaries with their citations, the conflict reviews and the reviewers' decisions, and the summary jobs and generation attempts.

Everything for one record (document, revision, company links, text units, embeddings, analysis status and event decisions) is written in a single transaction, so that a failed insert never leaves a partial article behind.

No transaction is held open while an embedding model or the LLM is running. Duplicates are handled by source identity: a changed payload becomes a new revision of the same document, while different accessions or article IDs stay separate documents even if their text is identical. Semantic similarity never deletes or merges stored records, it only links them to events. Old summaries and their citations are kept and never changed, so the history of each event stays auditable.

5.4 Event-Grouping

The event-grouping pipeline decides whether an incoming article or filing reports an event that is already tracked. The text is split into units, embedded, and compared with recent coverage of the same company; only plausible candidates are then shown to the LLM, which labels the pair as a repeat, an update, a conflict, a different event or uncertain. A label that is not backed by verbatim evidence on both sides is downgraded to uncertain and is kept as a pending link rather than being merged, and events are never joined transitively, so that one wrong link cannot chain unrelated events together. A filing is handled in the same way, except that it is split by Item so that each substantive Item can be grouped on its own.

When a piece of financial news is first reported and enters our system, a “No similar coverage” rule is applied to that article and a new event is generated. As the first follow-up articles is received, 2 scenarios are possible: if it is an exact copy (400 or more identical characters) of the first article, it will be tagged to the same event as a repeat, with no new summary call made, while if it refers to the same event but provides new information, it will be tagged as an update to that event, with a new event summary provided; once a second document joins, the event is labelled as corroborated. On the other hand, if a new article provides conflicting information to the financial event, the contradictory claim is detected and the article is labelled as a conflict, directing human review as the next step: a person decides whether the reports really conflict, and no source is declared right. If the model's label is not backed by verbatim evidence from both articles, the label is downgraded to uncertain: the article is stored as a pending link and is not merged into the event. Lastly, when an official SEC 8-K filing is received, it is compared with existing event records in the same way and joins the matching event as another source; the filing is the company's own disclosure, so it is not treated as confirmation of what the news reports.

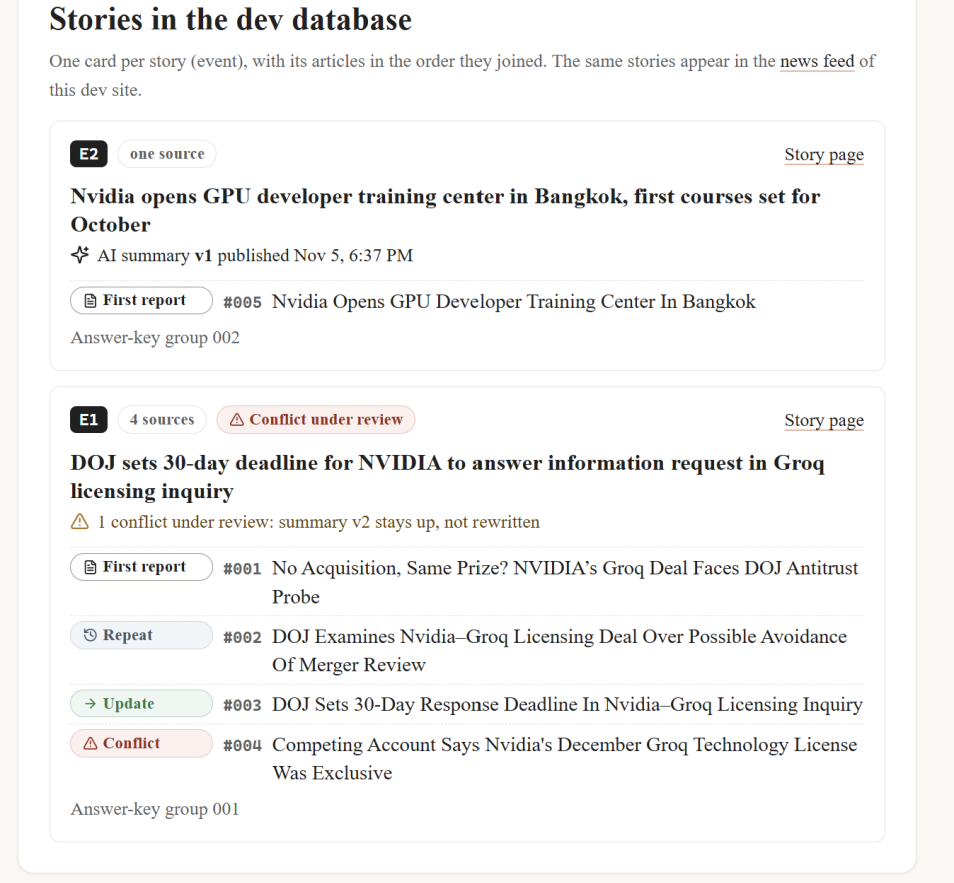


Figure 1. *Stories in the developer database: E1 holds four sources with a conflict under review, E2 a single source.*

5.5 Event Summarization & Conflict Resolution Architecture

For an event that is new or has received an update, the summary stage builds the evidence only from the passages of the event's active memberships, asks the LLM for a structured summary (headline, overview and key points), and validates it before publication. Every statement must cite passages with quotes that occur verbatim in them, every number in a statement must appear in the cited text, and advice phrasing is rejected. Validation establishes traceability to the sources rather than truth. Output that fails validation is stored as an invalid attempt and is never published. Publication supersedes the previous version in one transaction while old versions and their citations are kept, and an unchanged input makes no new model call; a repeat leaves the existing summary in place.

When the grouping stage flags a conflict, the published summary stays up with an under-review flag and no replacement is published until a person has reviewed it from the command line. The reviewer chooses one of four decisions: not_a_conflict, where the claims are compatible and the summary reports them normally; confirmed_conflict, where the regenerated summary must state both claims, each attributed and citing both sides, and declares no source to be right; different_events, where the incoming article is detached from the event and given an event of its own; and insufficient_evidence, where nothing is decided and the review keeps blocking publication. Every decision is recorded with the reviewer and explanation.

5.6 Project Scope & Limitations

5.6.1 Functional scope

The project scope is bounded to ensure precision and maintain manageable computational overhead during real-time processing. By default the system monitors NVIDIA; the supplied configuration also lists 11 U.S. large-cap companies, including NVDA, MSFT, AAPL, GOOGL and AMZN, which were used as a scalability test. Broader market coverage, micro-cap scraping and unrelated market news are intentionally excluded to control computational cost (API and tokens), data quality, and maintaining focus on the overall project aim. Articles on price prediction, sentiment trading signals, or automated trade execution are also omitted for similar reasons. The system performs cross-source entity resolution and deduplication to categorise inputs under distinct events, instead of treating every document as standalones, and provides explainable relevance and source provenance via a dedicated web application for transparency. The system also strictly disclaims objective "truth scores," guaranteed verification, opaque impact metrics, portfolio optimization, and buy/sell recommendations.

5.6.2 System Limitations

In light of the project scope mentioned above, the following constraints have been implemented. The extraction of SEC filings is limited to the document text within the primary SEC Form 8-K; associated exhibit files are preserved as URL references instead of being downloaded and parsed. The temporal nature of 8-K filings signals a delay between the actual major event and its reporting to the SEC, meaning our system does not rely on the filings for immediate reporting. There is also a recovery window for pulling secondary news articles, 72 hours by default and 3 hours in the supplied .env.example, and a 40-page REST limit, which helps address rate-limiting blocks from the Alpaca API and provides a manageable bound for synchronisation. The hybrid retrieval architecture combining vector embeddings and lexical full-text search is also bounded by operational windows and the constraints of the pre-trained BAAI/bge-small-en-v1.5 model. The automated verification used for event summaries establishes traceability, not truth: it checks that every quote occurs verbatim in its cited passage and that every number appears in the cited text, but it does not prove that a statement is fully supported by its quotes, so it does not guarantee that a summary is accurate.

6. System Development & Implementation

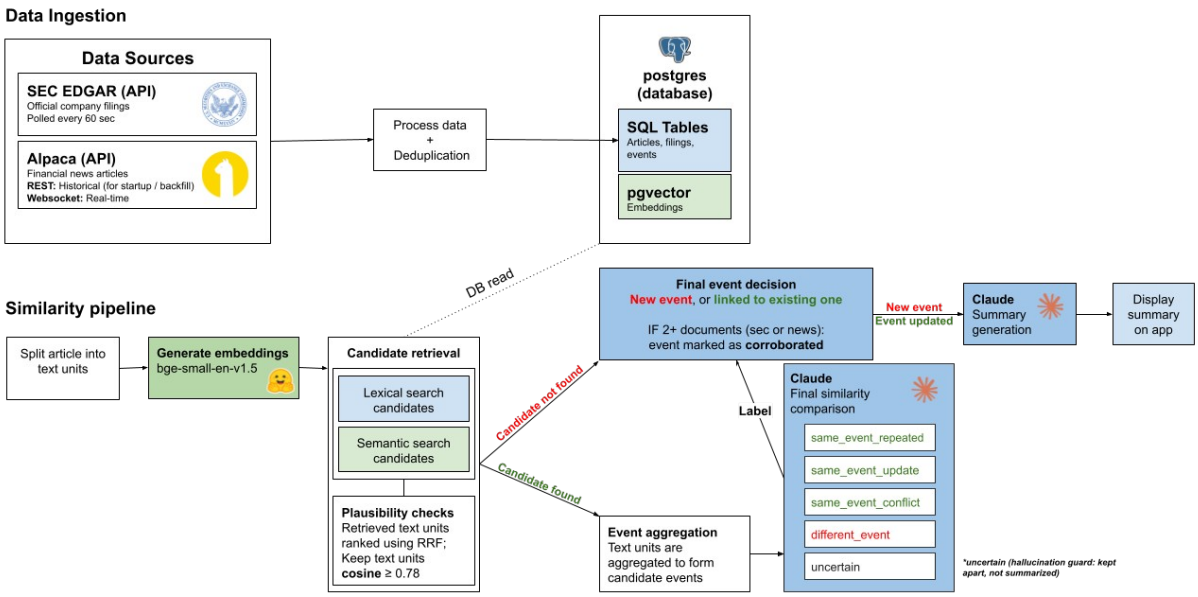


Figure 2. *System architecture diagram.*

In the similarity pipeline, light blue indicates a query of the postgres tables, blue indicates an update/insert to the postgres tables. Likewise, light green indicates a query of the pgvector embeddings in postgres, and green indicates an update/insert to the pgvector embeddings.

As a whole, our project runs as a local application. The user must start up the application to refresh the news and start the process for real-time data streaming, event grouping, and summarization. All data is stored in the user's local database. Additionally, the user must provide their own API keys.

Data Sources. We are using two APIs as our data sources. For official company filings with the SEC, we are using the SEC EDGAR API. This API is polled every 60 seconds for updates. For financial news articles from Benzinga, we are using the Alpaca API. Alpaca provides both a REST API, which provides historical news data, as well as a websocket that can be used for real-time data streaming. When the user first starts up the web server, a backfill process queries both SEC EDGAR and the Alpaca REST API. After the backfill process is complete, the application runs in the background with the real-time data collection by polling the SEC EDGAR API every 60 seconds and waiting for new events from the Alpaca websocket.

Local data storage. The data is stored locally, and requires a user to set up a local postgres database. A database named "stock_news" houses all the SQL tables for the application. Pgvector within postgres is used for storing the embeddings.

6.1 Similarity pipeline

When new articles or filings are collected from the APIs, they flow through our similarity pipeline. After the articles go through our standard data processing and deduplication checks, they are stored in a postgres table. To group news articles into unique events, our similarity pipeline checks the content of the articles through embedding → candidate retrieval → plausibility checks → Claude as the final judge for similarity comparison → final decision regarding the event.

Text preparation. Each incoming article or filing is processed into segments, which contain one or more text units. Text units are the chunks used for embedding and retrieval, while segments are the units for which the pipeline determines event relationships.

Embeddings. The text units undergo embedding using HuggingFace's bge-small-en-v1.5 model. These vectors are stored using pgvector in postgres.

Candidate retrieval. For each article, which has been split into text units, we identify potentially similar events that already exist in our database through candidate retrieval. We use two different methods: semantic search and lexical search, then combine the candidates and rank using reciprocal rank fusion.

Semantic search. The embeddings of the incoming article's text units are compared against embeddings of existing text units in the database using cosine similarity, calculated in SQL. For each incoming text unit, the top `semantic_top_k` matching text units are returned, ranked by similarity (default = 20).

Lexical search. PostgreSQL full-text search compares the incoming text units against indexed text units in the database using word-based matching. A lexical relevance score is calculated, and the top `lexical_top_k` hits are returned (default = 20).

Reciprocal rank fusion (RRF). The semantic and lexical search results are combined into a single ranking using reciprocal rank fusion. Since the raw scores from the two retrieval methods use different scales, RRF combines their rankings rather than their raw scores. Each text unit receives a contribution of (1/(k+rank)) from both the semantic and lexical search rankings, where k = 60 (default) and rank starts at 1. These contributions are summed to produce the fused score, with higher scores ranking the text unit more highly.

Event aggregation and plausibility checks. After RRF, the system looks up the existing event memberships associated with the retrieved text units. Each retrieved text unit must pass the plausibility check before it can contribute to a candidate event. By default, a retrieved text unit passes if its semantic cosine similarity is at least `SIMILARITY_MIN_SEMANTIC_SIMILARITY` (default = 0.78). Lexical search contributes to ranking but does not independently qualify a retrieved text unit by default. Qualifying retrieved text units are then aggregated by existing event ID to form candidate events. The system ranks these events by their best fused score and limits the shortlist before sending relevant candidates to Claude for comparison, reducing unnecessary model calls.

6.1.1 Claude for final comparison

Claude performs the final similarity comparison between an incoming article segment and a shortlist of existing candidate events identified by the retrieval pipeline. Unlike semantic and lexical search, which

identify potentially related text, Claude evaluates whether the incoming information actually describes the same real-world event. The exact Claude prompt can be found in `SystemCode/src/datasources/similarity/comparison.py`.

Comparison criteria. Claude compares the incoming segment with each candidate event's evidence to determine whether they describe the same real-world occurrence.

- **Participants, actions and objects:** Are the companies or people involved, the action taken, and the subject of the event consistent?
- **Event date and fiscal period:** Do the passages refer to the same occurrence or reporting period? Publication dates alone do not determine whether events match.
- **Figures and metrics:** Are the values, units, currencies, metrics and periods comparable and consistent?
- **Event status and developments:** Does the incoming information repeat existing facts, add new information, or report a later development, such as a transaction being completed or cancelled?
- **Conflicts and uncertainty:** Does a genuinely comparable claim contradict existing evidence, or is there insufficient information to decide? Missing information is not a contradiction, and Claude does not determine which source is correct.

Table 1. *Relationship labels output by Claude*

Label	Meaning
same_event_repeated	The incoming information repeats information about an existing event.
same_event_update	The information adds new details or reports a later development concerning the same event.
same_event_conflict	The information concerns the same event but contains an incompatible claim that requires conflict handling.
different_event	The incoming information describes a different event from the candidate.
uncertain	The available evidence is insufficient to determine the relationship confidently.

How the result is used. Claude's output is used to determine the next step. A confirmed match links the incoming segment to an existing event. An update can trigger summary regeneration, while a conflict can require review. An uncertain relationship remains pending rather than being treated as a confirmed match. If all relevant candidates are rejected and no unresolved comparisons or processing failures remain, the system can create a new event.

If no matching candidates are found during retrieval, the article segment is not sent to Claude, and a new event is created.

If a second document from any source joins an event, the event can be marked as “corroborated”. SEC filings are also incorporated as an additional source of information to provide further context.

6.1.2 Summary generation

If a new event is created or the event is updated, Claude is called again to generate a summary for the event. The exact Claude prompt can be found in `SystemCode/src/datasources/summary/generation.py.`

Claude is prompted to generate a short, neutral, evidence-based summary for a financial news event using only the provided news articles and SEC filing content. It produces a factual headline, a one to three sentence overview, and up to three additional key points without repeating information. Every statement must be supported by cited source passages, with short verbatim quotes as evidence.

The prompt also enforces strict factual accuracy: figures, dates, fiscal periods, and status words must remain faithful to the sources. Additionally, missing information must not be guessed, and claims must be attributed to their sources. The output must remain neutral and avoid investment advice or opinions.

Both models are set in the .env file. The supplied .env.example uses Claude Sonnet 4.5 for consolidation and Claude Opus 5 for summaries.

6.2 App Screenshots

The homepage for a given stock shows unique events, consolidating news for the same event into one. The AI summary provides users the most important details of the event.

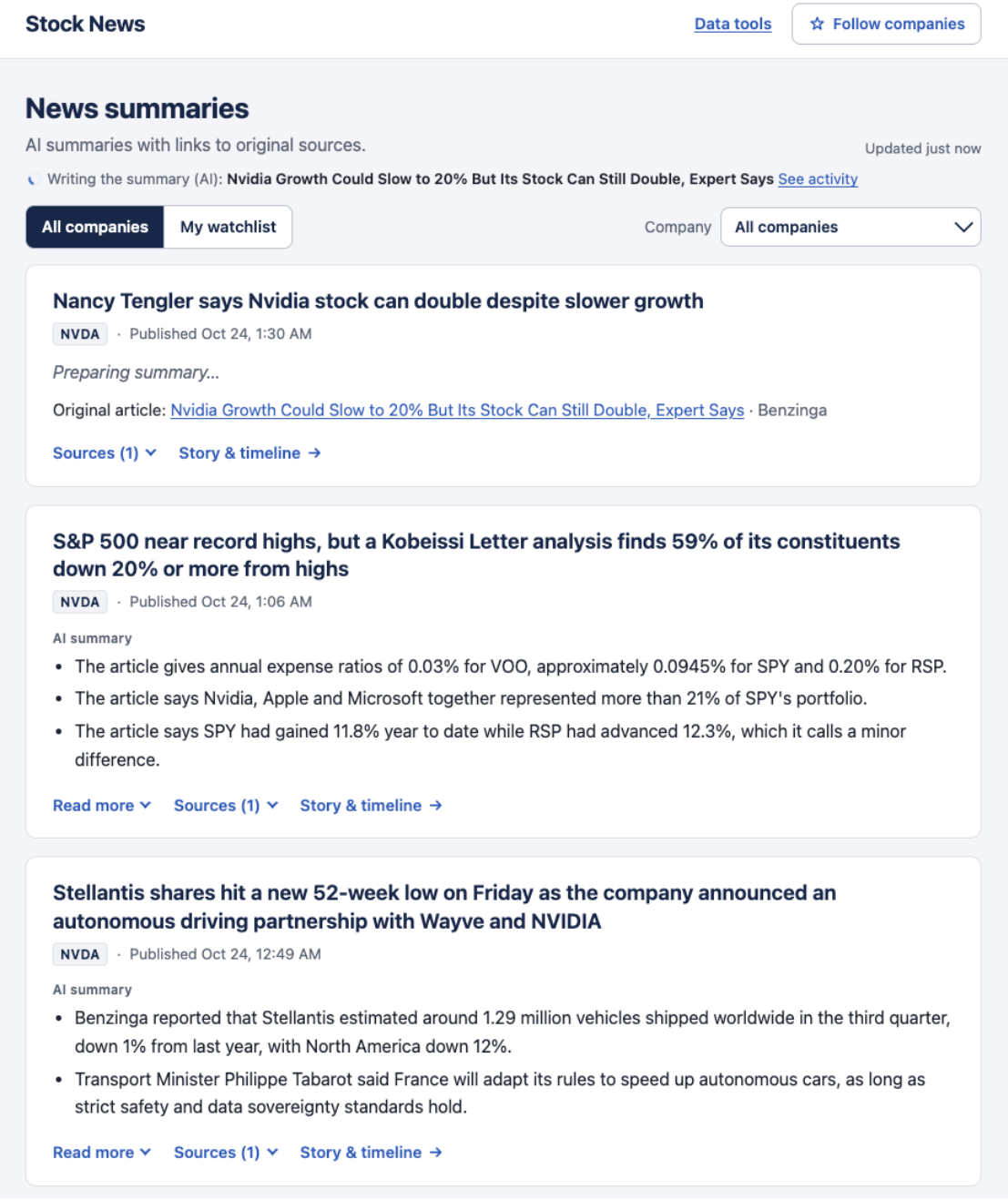


Figure 3. *Stock homepage with consolidated events and AI summaries.*

Upon selecting an event, the summary, a timeline of the event, original sources, and supporting evidence for the summary is provided.

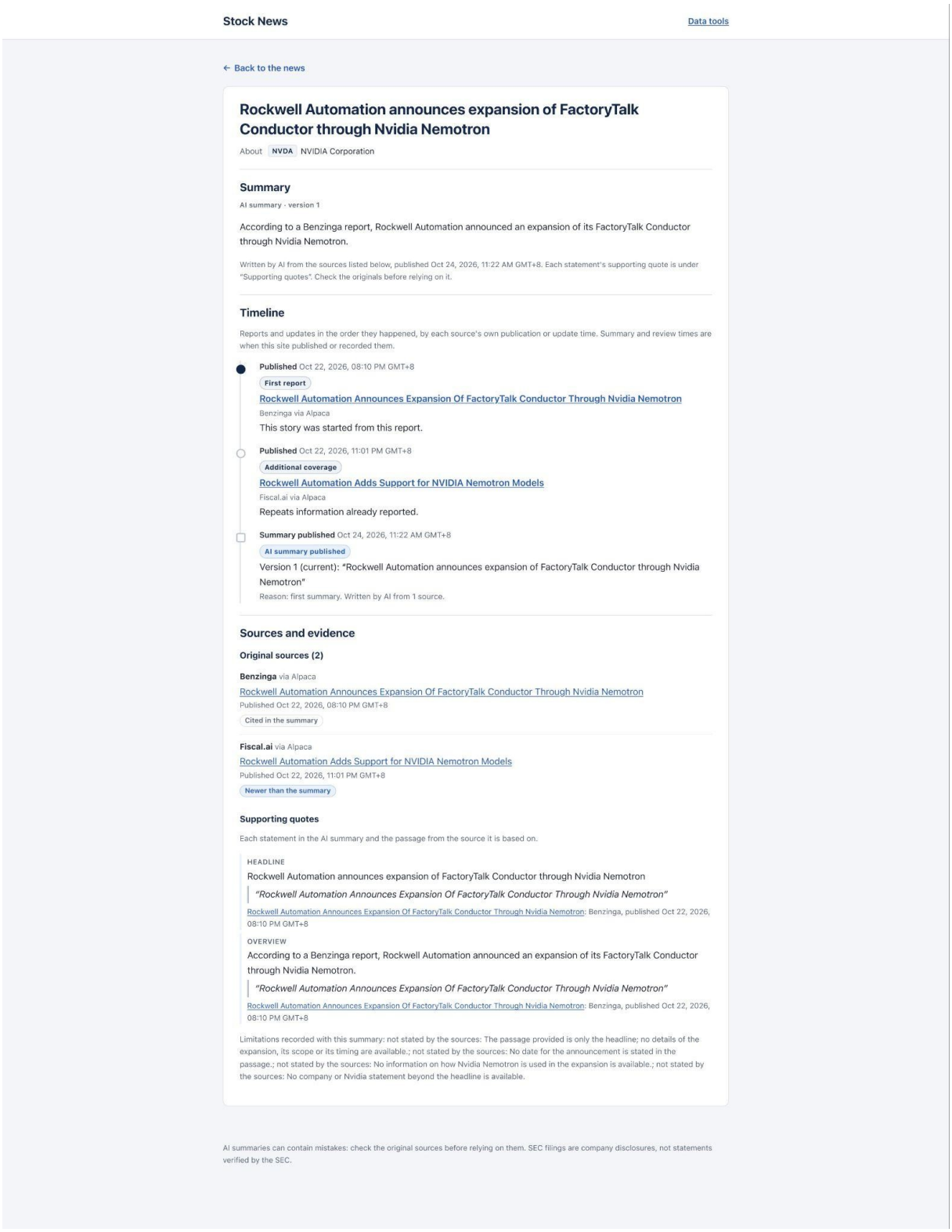


Figure 4. *Event detail view with summary, timeline, original sources and supporting evidence*

7. Experiments

7.1 Objectives and research questions

StockEvent Lens handles news in two steps (Figure 5). **Consolidation** groups incoming articles into events and labels each one a *repeat*, an *update* or a *conflict*; an article about something else opens a *different* event. **Summarization** then writes one summary per event and flags conflicting reports instead of merging them. This section evaluates consolidation; summarization is not scored here.

Three research questions guide the evaluation:

RQ1 – Labelling accuracy. How accurately does the pipeline label each article, and how much does the LLM judge matter? Measured by macro-F1 against a no-LLM rule baseline; answered in Section 8.1.

RQ2 – Error origin. Do the remaining errors come from retrieval, the prompt, the model's reasoning or the evidence check? Every error of two API judges is traced to its cause; answered in Section 8.2.

RQ3 – Blind spots. What is a similarity-threshold-plus-LLM design most likely to miss? Each recurring error is linked to batches and quoted text; answered in Section 8.3.

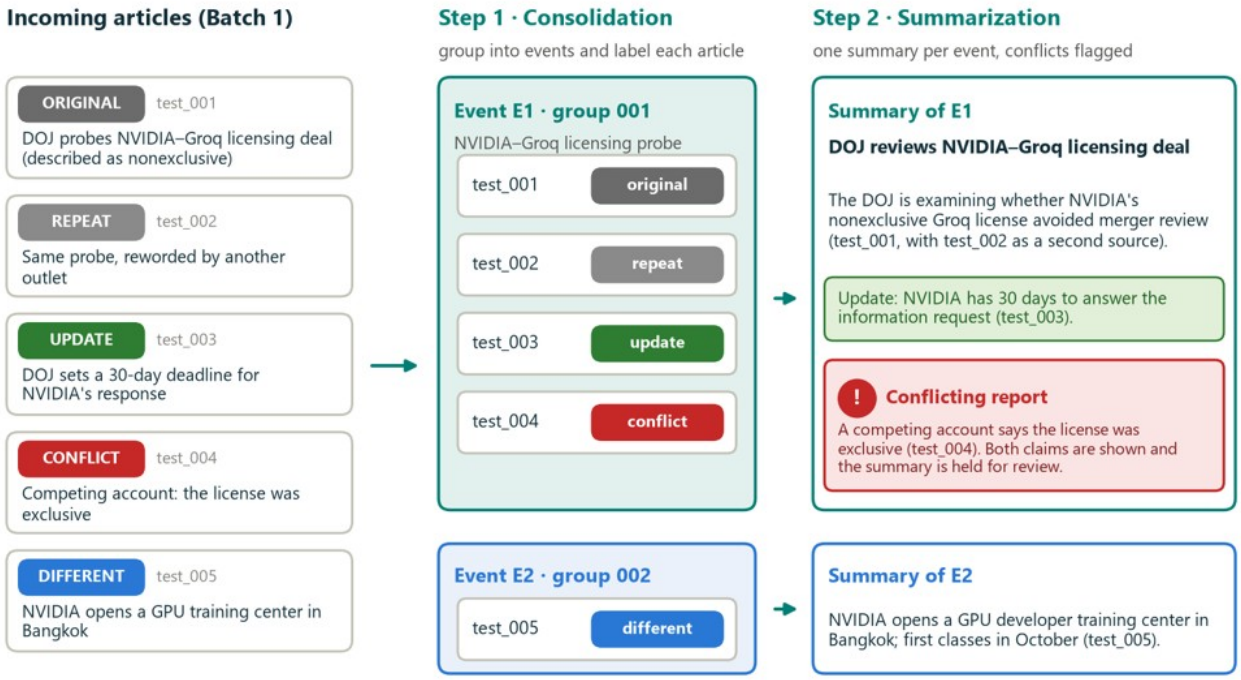


Figure 5. *Consolidation and summarization on Batch 1 (test_001–test_005): five articles become two events, and the summary keeps the update and flags the conflict*

7.2 Labeled synthetic data activity

Each of the five members contributed 25 articles in five batches of five, giving **125 articles in 25 batches**. Every batch follows the pattern of Figure 5: one real original, three variants of the same event (repeat, update, conflict) and one different-event article, so the dataset covers **50 distinct events.** The originals are real NVIDIA news, 24 from Benzinga and one from CBS News. All conflicts are

synthetic, usually with one changed fact such as $10B → $5B; repeats and updates are each 20 synthetic and 5 real, and 15 of the 25 different-event articles are real. In total 75 articles are synthetic and 50 real, published on 20 dates between 10 September and 9 October 2026. Expected labels come from the stored answer key.

7.3 Experimental procedure

Each batch ran in its own **temporary database**, because the answer key only relates articles within a batch; this keeps the ground truth clean but is **easier than production**. Articles went through the production pipeline in order: embedding with bge-small-en-v1.5 in passages of about 290 words, hybrid retrieval, a cosine-similarity filter at 0.78, the LLM judge, which sees **at most three passages** per candidate event, and the evidence check. The first article of each batch opens the event, so **100 articles were scored**. Table 2 lists the six configurations.

Table 2. *Configurations compared*

Configuration	Run by	Judge	Retrieval settings	Status
Rule baseline	Nathan	No LLM: always answers *repeat*	Up to 3 candidate events	Complete
Local LLM	Nathan	**Qwen2.5-7B-Instruct** (Ollama)	Lexical route on; up to 6	Complete
Claude Haiku	Claire	**Claude Haiku 5.5** (Anthropic)	Lexical route on; up to 6	Complete
Claude Sonnet	Nakhin	**Claude Sonnet 5.5** (Anthropic)	Up to 3 candidate events	Complete
Gemini	Nghia	**Gemini 3.8 Flash** (Google)	Up to 3 candidate events	Complete
Claude Opus	Hao	**Claude Opus 5.5, low effort** (Anthropic)	Lexical route on; up to 6	Complete

The **rule baseline** labels every compared article a repeat, so retrieval alone decides the grouping; it is the floor any LLM must beat. The runs used two retrieval settings, so close results across settings are not directly comparable. In a separate **direct-prompting review**, Gemini read each complete batch with no retrieval limits and wrote the event summaries; it knew the batch structure, so this review is qualitative.

7.4 Metrics

Macro-F1, the primary metric, averages the F1 of the four labels, so a weak label cannot hide behind an easy one. *Label accuracy* counts exact matches, with uncertain and failed articles counted as wrong. *Pairwise grouping F1* checks whether each pair of articles in a batch is correctly placed together or apart; it mostly measures retrieval. With about 25 articles per label, differences below roughly 15 points may be noise. **Test articles**:
https://github.com/Qwenty228/GC1/tree/main/SystemCode/populate/event_grouping.

Per-article results:
https://github.com/Qwenty228/GC1/tree/main/SystemCode/evaluation/consolidation/results.

8. Findings and discussion

8.1 LLM performance on similarity comparison (RQ1)

Table 3. *Consolidation results on the 100 scored articles (best LLM value per row in green)*

Metric	Rule baseline	Local Qwen2.5-7B	Claude Haiku 5.5	Claude Sonnet 5.5	Gemini 3.8 Flash	Claude Opus 5.5
Label accuracy	*48.0%*	46.0%	79.0%	87.0%	88.0%	**92.0%**
Macro-F1 (primary)	*35.5%*	45.3%	80.3%	86.9%	88.9%	**92.0%**
Grouping precision / recall	*94.7% / 96.0%*	100% / 47.3%	100% / 77.3%	100% / 86.0%	100% / 88.7%	**100% / 90.0%**
Grouping F1	*95.4%*	64.3%	87.2%	92.5%	94.0%	**94.7%**
Uncertain / failed articles	*0 / 0*	17 / 0	4 / 0	**0 / 0**	2 / 0	**0 / 0**

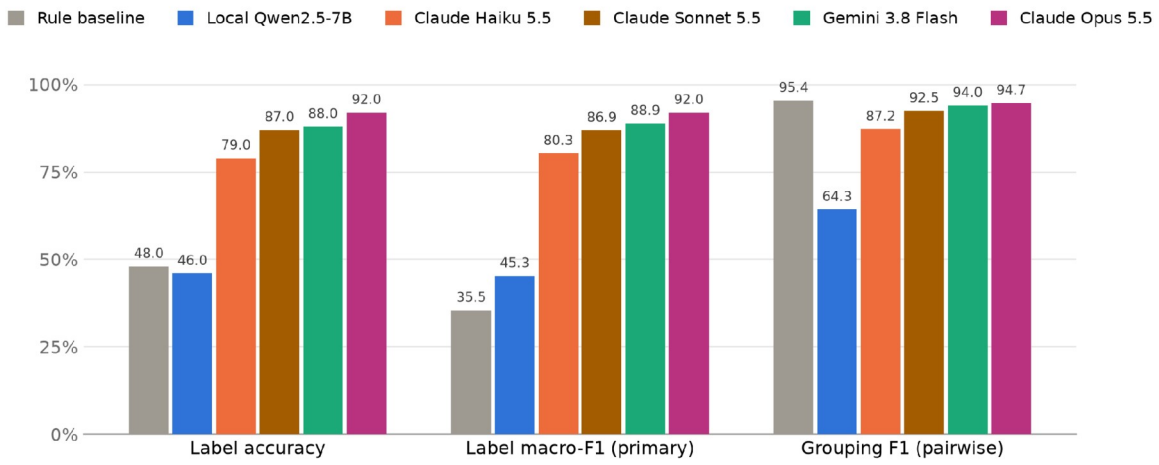


Figure 6. *Label accuracy, macro-F1 and grouping F1 by configuration (grey: rule baseline, no LLM)*

Finding 1: Every API judge clearly beats the local model and the rule baseline. Macro-F1 ranges from 80.3% for Claude Haiku to **92.0%** for Claude Opus, against 45.3% for local Qwen and 35.5% for the rule baseline, and every API judge is ahead of Qwen with $p < 0.001$. Gemini and Haiku do not differ significantly (exact McNemar $p \approx 0.06$).

Finding 2: Grouping F1 does not separate the models. No LLM merged two different events, and the rule baseline scores highest on grouping because retrieval alone places most articles correctly.

Table 4. *Gemini 3.8 Flash confusion matrix (rows: expected; columns: predicted)*

Expected \ Predicted	Repeat	Update	Conflict	Different	Uncertain	Recall	F1

Repeat	**22**	3	0	0	0	88.0%	91.7%
Update	1	**19**	0	4	1	76.0%	**77.6%**
Conflict	0	2	**22**	0	1	88.0%	93.6%
Different	0	0	0	**25**	0	100.0%	92.6%
Precision	*95.7%*	*79.2%*	*100.0%*	*86.2%*	–		

Table 5. *Claude Haiku 5.5 confusion matrix (rows: expected; columns: predicted)*

Expected \ Predicted	Repeat	Update	Conflict	Different	Uncertain	Recall	F1
Repeat	**17**	5	0	0	3	68.0%	79.1%
Update	1	**15**	0	8	1	60.0%	**63.8%**
Conflict	0	2	**22**	1	0	88.0%	93.6%
Different	0	0	0	**25**	0	100.0%	84.7%
Precision	*94.4%*	*68.2%*	*100.0%*	*73.5%*	–		

Finding 3: Updates are the hardest label. Update F1 is 77.6% for Gemini and 63.8% for Haiku; Haiku split 8 of 25 updates into new events, Gemini 4. Every conflict either model flagged was genuine. Qwen failed mainly on repeats, labelling 16 of 25 as updates (repeat F1 14.8%), and left 4 requested comparisons out of its answers, which count as wrong.

8.2 Where the errors come from (RQ2)

Every error of Gemini and Haiku was traced to its cause from the stored explanations, validator notes and code (Table 6). Both used the same judge prompt.

Table 6. *Where each error originates, with its code location (numbers are test article IDs)*

Where the cause is	What happened	Gemini 3.8 Flash	Claude Haiku 5.5
Retrieval window *retrieval.py:205*	Follow-up published more than 7 days after every earlier article in its batch, so no candidate was found and the LLM was never called	**2** · 078, 083	**2** · 078, 083
Partial view of the original *retrieval.py:322, 378*	"New" fact was in the original, beyond the passages the judge sees	**2** · 037, 072	**2** · 022, 072
Prompt vs key: dates *comparison.py:163–165, 175*	Next-day or later-dated follow-up split into a new event	**2** · 058, 068	**6** · 028, 058, 068, 073, 074, 093
Prompt vs key: reversals *comparison.py:171, 184*	Canceled or reversed plan judged an update, keyed as conflict	**2** · 029, 049	**2** · 049, 064
Prompt vs key: materiality *comparison.py:168–171*	Extra analysis counted as an update	**0**	**1** · 102

LLM reasoning	Restated detail treated as new; new facts missed (118); topic misjudged; unsure	**2** · 027, 118	**5** · 007, 043, 052, 118, 122
Evidence validator *comparison.py:367–385*	Quote not found verbatim, label downgraded to *uncertain* (4 of 5 labels were correct)	**2** · 009, 063	**3** · 067, 107, 113
Total		**12**	**21**

Amber: the model followed the prompt, but the prompt and the answer key disagree. Red: the model's own reasoning. Code paths are under SystemCode/src/datasources/similarity/.

Finding 4: Most errors are caused by the pipeline, not by the model's judgement. Only 2 of Gemini's 12 errors and 5 of Haiku's 21 are reasoning mistakes. The rest happened because the article never reached the model, the model saw only part of the original, our prompt contradicted the answer key, or a correct label was rejected because a quote did not match word for word.

8.3 What the threshold-plus-LLM design is most likely to miss (RQ3)

Four blind spots stand out; Table 7 gives the evidence for each.

Late follow-ups published more than 7 days after the original fall outside the search window and never reach the judge, even when the text is similar.

Facts outside the judge's view: in 7 of 8 repeat-to-update errors the "new" fact was already in the original, often beyond the three passages the judge sees.

One-number conflicts look like repeats to the similarity filter, so only the LLM and exact quote matching catch them.

Continuations and reversals depend on day words: the prompt treats different trading sessions as separate events and cancelled plans as updates, so test_058, Thursday's recovery after Wednesday's fall, is split from its original.

Table 7. *Traceable evidence for each blind spot (batch, event group, test files and quoted text)*

Blind spot	Batch · group	Original → incoming	Evidence in the articles	Gemini	Claude Haiku
Late follow-ups	16 · 031	test_076 → test_078	test_078 was published on 2 October, 7 days 5 h after test_077; text similarity 0.89, above the 0.78 filter	different (no LLM call)	different (no LLM call)
	17 · 033	test_081 → test_083	test_083 was published on 2 October, 7 days 7 h after test_082; text similarity 0.93	different (no LLM call)	different (no LLM call)
Facts outside the judge's view	8 · 015	test_036 → test_037	"trading 2.46% higher at $236.53" is at word 525 of 572 in test_036	update	repeat
	15 · 029	test_071 → test_072	"a draw of 2.090M barrels" is at word 799 of 929 in test_071	update	update
	5 · 009	test_021 → test_022	Enflame and H200 details first appear at words 262 and 312 of 476 in test_021	repeat	update

Next-day continuations	12 · 023	test_056 → test_058	"down about 6% on Wednesday" → "recovered … on Thursday … after Wednesday's sector-wide decline"	different	different
	14 · 027	test_066 → test_068	"tumbling Wednesday" → "recovered on Thursday … after Wednesday's sell-off"	different	different
	6 · 011	test_026 → test_028	"a post on X on Thursday" → "announced late Friday"	update	different
	15 · 029	test_071 → test_073	"Treasury Auction To Test Sentiment" → "auction drew strong demand on Wednesday"	update	different
	15 · 029	test_071 → test_074	Claude: "$25 billion auction on Tuesday; candidate … $39 billion auction … on Wednesday"	conflict	different
	19 · 037	test_091 → test_093	test_093: "follow-up to the September 25 valuation comparison … for September 28"	update	different
Reversals	6 · 011	test_026 → test_029	"over 250kW of solar power" → "capping … at a meager 45 kW"	update	conflict
	10 · 019	test_046 → test_049	"transfer about $8 billion" → "completely abandoned plans"	update	update
	13 · 025	test_061 → test_064	"borrow about $40 billion" → "instead raise about $8 billion in new equity"	conflict	update
One-number conflicts and quote check	2 · 003	test_006 → test_009	"up to $10 billion" → "was $5 billion"	uncertain: quote rejected	conflict
	13 · 025	test_061 → test_063	"preparing to borrow about $40 billion" → financing finalized	uncertain: quote rejected	update

Batch = (test number − 1) ÷ 5 + 1; the event group is in each file name. Green: correct; red: incorrect.

Finding 5: With full context, Gemini kept what the pipeline missed. Reading whole batches, it kept the update and flagged the conflict in all 25 batches by its own checklist. The review also found one dataset defect: test_078's headline contradicts its body.

8.4 Discussion and limitations

Impact and improvements. Consolidation errors carry into the summaries: a missed update splits one development across two summaries, and a conflict read as an update loses its flag. The fixes follow Table 6: show the judge the full earlier article, align the prompt and answer key on continuations and reversals, widen the 7-day search window for slow-moving stories, tune the 0.78 threshold on held-out batches, allow near-verbatim quote matching, and audit synthetic articles such as test_078.

Limitations. The batches are small, controlled, largely synthetic and scored independently, with about 25 articles per label and one run per model at two retrieval settings; the direct-prompting review is qualitative. The comparison models the system ships with, Claude Sonnet 4.5 in .env.example and Claude Opus 5 as the code default, were not evaluated, and summarization was scored only for local models and is not reported here.

Key takeaway. API judges (Gemini 3.8 Flash, Claude Sonnet 5.5, Claude Haiku 5.5, Claude Opus 5.5) reached **80–92% macro-F1 with zero false merges**; most remaining errors come from *what the judge is shown*, *what the prompt instructs* and *how quotes are verified*, so the pipeline, not the model, is where improvement should start.

APPENDICES

Appendix of **report**: Mapped System Functionalities against knowledge, techniques and skills of modular courses: MR, RS, CGS

Table A1. *System functionalities mapped to MR, RS and CGS*

Course	Knowledge / technique	Where it is used in StockEvent Lens	What it demonstrates
MR Machine Reasoning	**Knowledge discovery and data mining**	Text embeddings (bge-small-en-v1.5) in pgvector; hybrid semantic + lexical (PostgreSQL full-text) retrieval fused by reciprocal rank fusion; grouping documents into events	Discovering which documents describe the same real-world development
	Decision automation (business rules)	Rules that decide without the LLM: cosine ≥ 0.78 filter, 7-day window, ≥ 400 identical characters → repeat, no candidate → new event; label-to-action rules (update → regenerate summary, conflict → hold for review, uncertain → pending)	Fast, consistent and explainable decisions for clear-cut cases; the LLM is used only for ambiguous ones
	Knowledge representation	Event records linking each article segment to an event with its relationship label, matching facts, differences and verified quotes; event status (planned, announced, completed, canceled, …); SEC 8-K filings as the primary source	Structured, traceable knowledge about each event and which sources support it
RS Reasoning Systems	**Hybrid reasoning**	One pipeline chaining data-driven retrieval, rule-based filters and LLM judgment; only a shortlist of 3 to 6 candidate events reaches the LLM	Using each technique where it is strongest while limiting LLM cost
	Reasoning under uncertainty	Explicit *uncertain* label; same-event judgments whose evidence quotes cannot be found verbatim are downgraded; conflicts are held for human review rather than auto-resolved	Avoiding over-confident automated decisions
	Evaluation of reasoning quality	125-article labeled test set; macro-F1, per-class precision and recall, pairwise grouping F1; rule baseline vs local and API LLM judges	Measuring reasoning quality objectively and locating failure modes
CGS Cognitive Systems	**Natural language reasoning with LLMs**	LLM judge compares participants, actions, dates, figures and event status to label repeat, update, conflict or different; interchangeable models (Claude, Gemini, local Qwen)	Human-like judgment of whether two texts describe the same event
	Grounded natural language generation	Event summaries (headline, overview, key points) in which every statement cites verbatim source quotes; neutral wording, no investment advice	Explainable output that users can verify against the original sources
	Processing unstructured text	Cleaning Benzinga HTML and SEC EDGAR filings; splitting documents into segments and text units for embedding	Turning raw news and filings into machine-readable knowledge

Appendix of **report**: Actual system screenshots

Figures A1 to A8 show the running system: the news feed, then Batch 1 (test_001–test_005) replayed in developer mode, one article at a time.

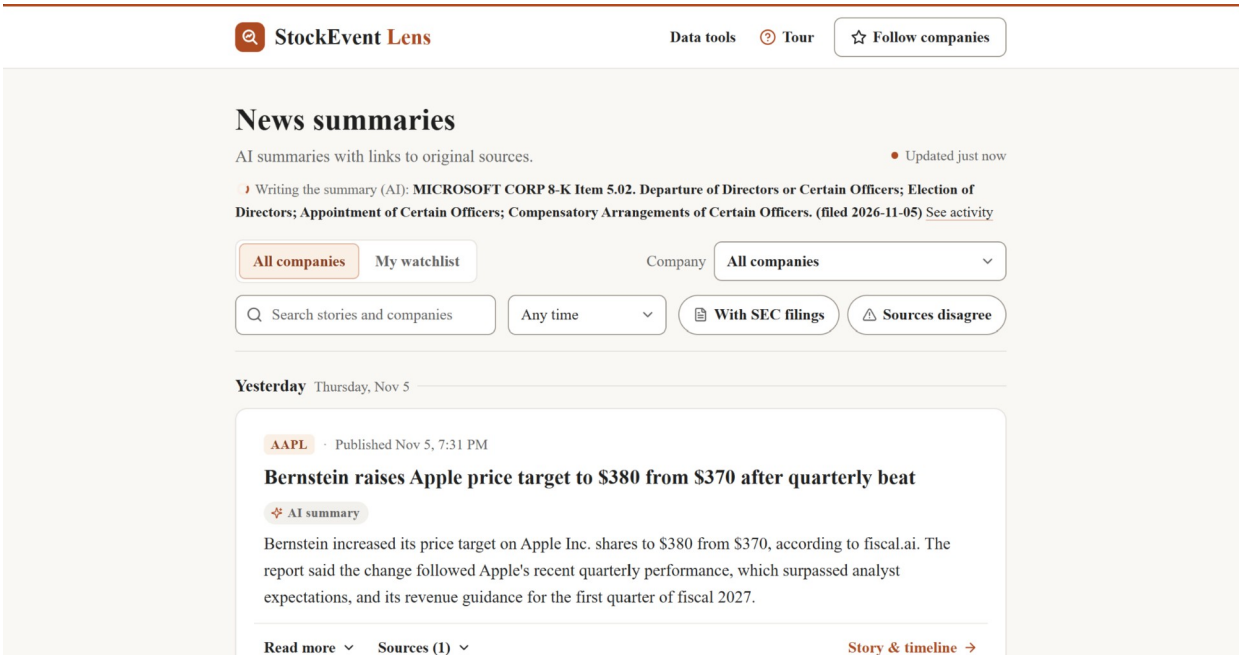


Figure A1. *The news summaries page: each story has an AI summary and links to its original sources*

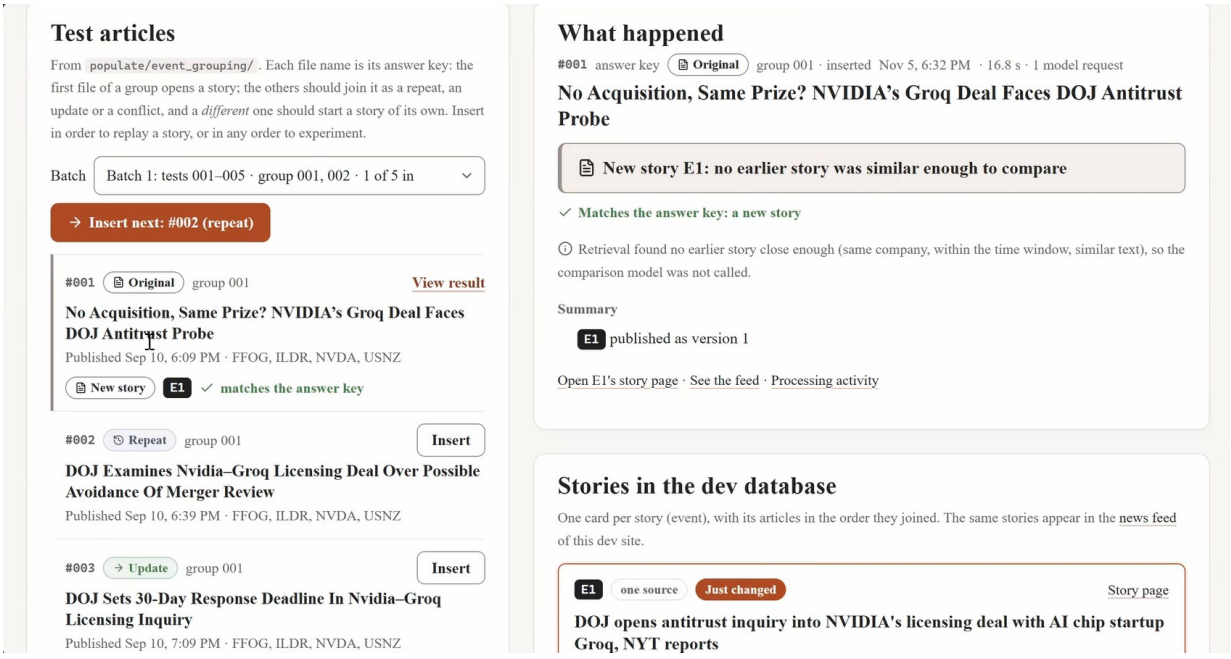


Figure A2. *Developer mode, batch 1: the first article opens a new story (E1)*

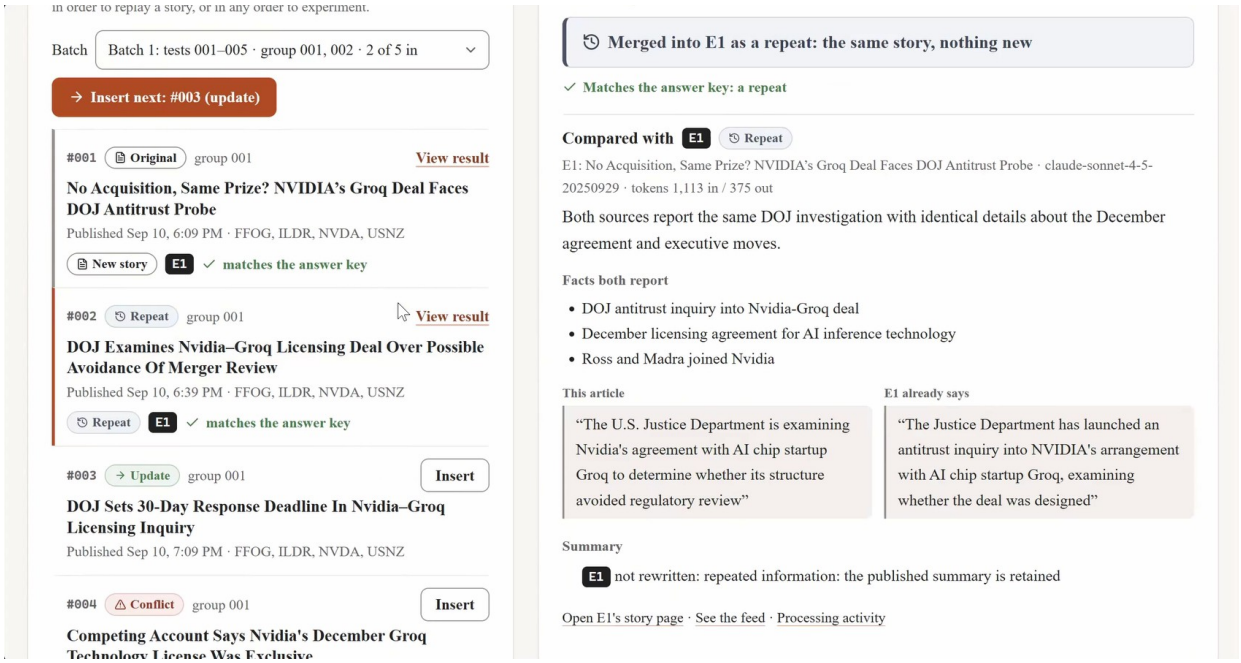


Figure A3. *The second article is merged into E1 as a repeat; the summary is not rewritten*

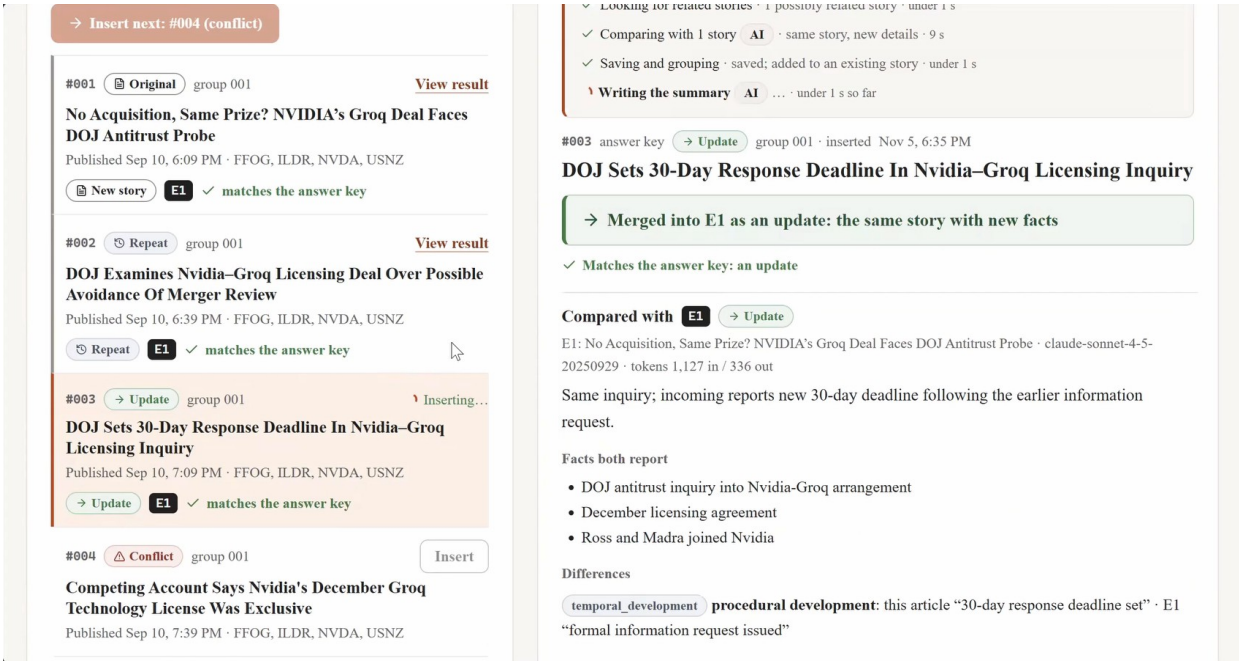


Figure A4. *The update is merged into E1 with its new fact; the summary becomes version 2*

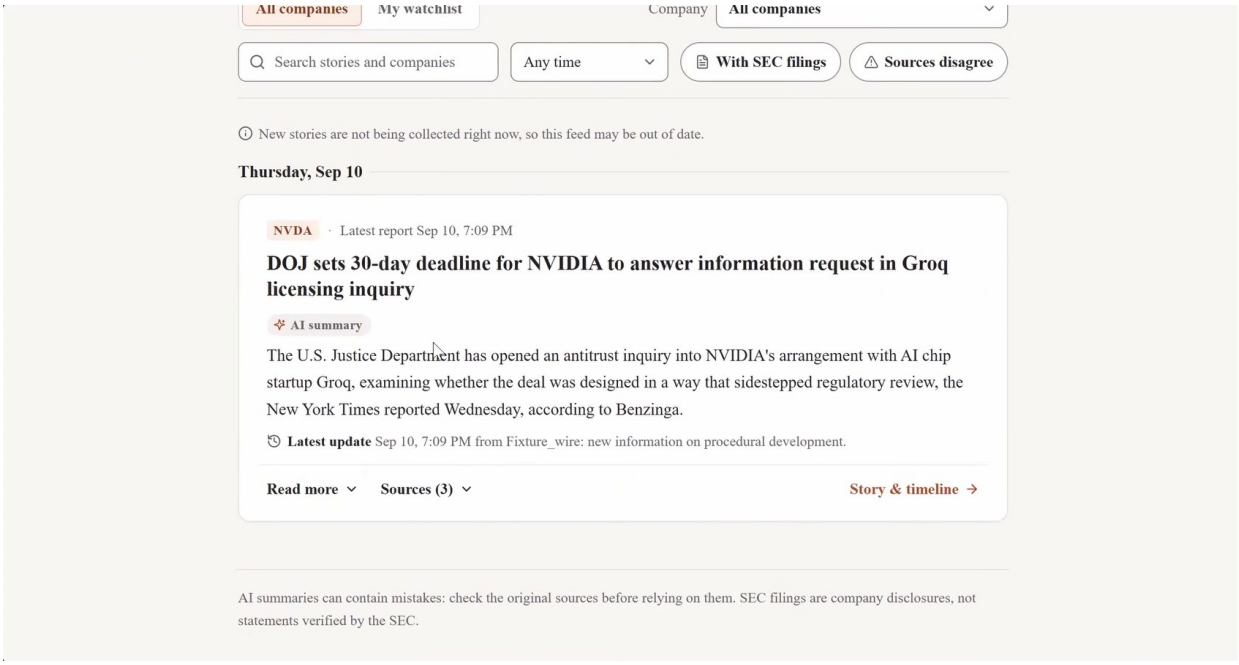


Figure A5. *The news feed: one story with three sources*

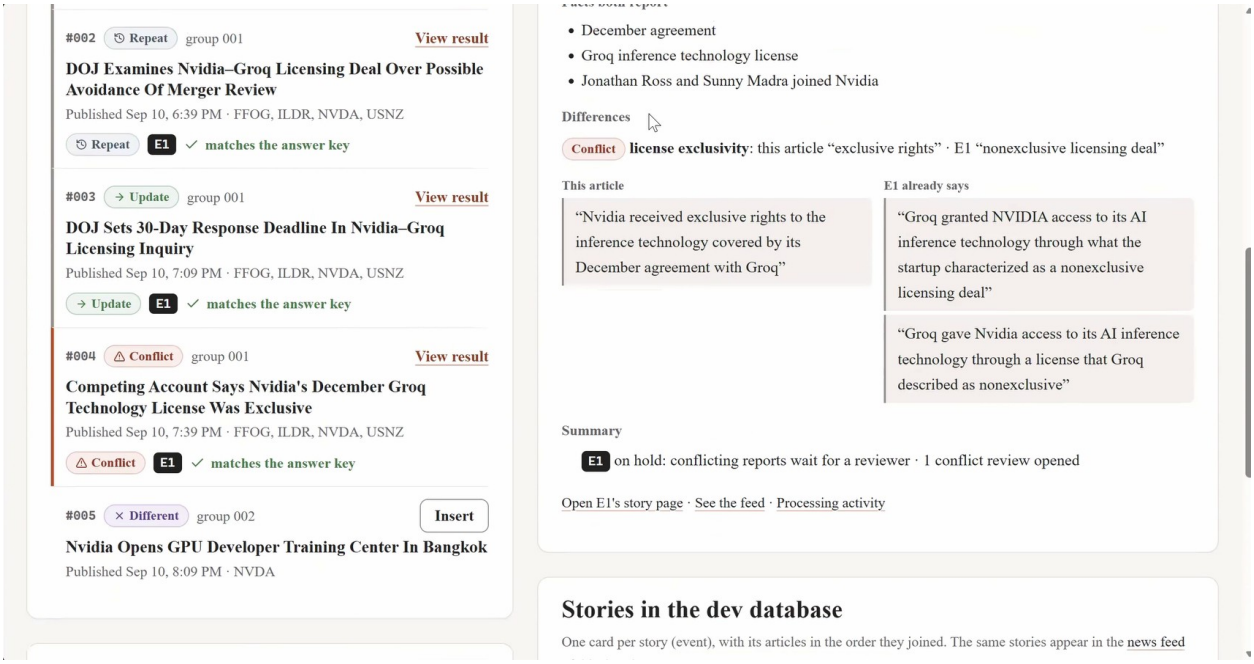


Figure A6. *The conflict: both quotes shown, the summary held for review*

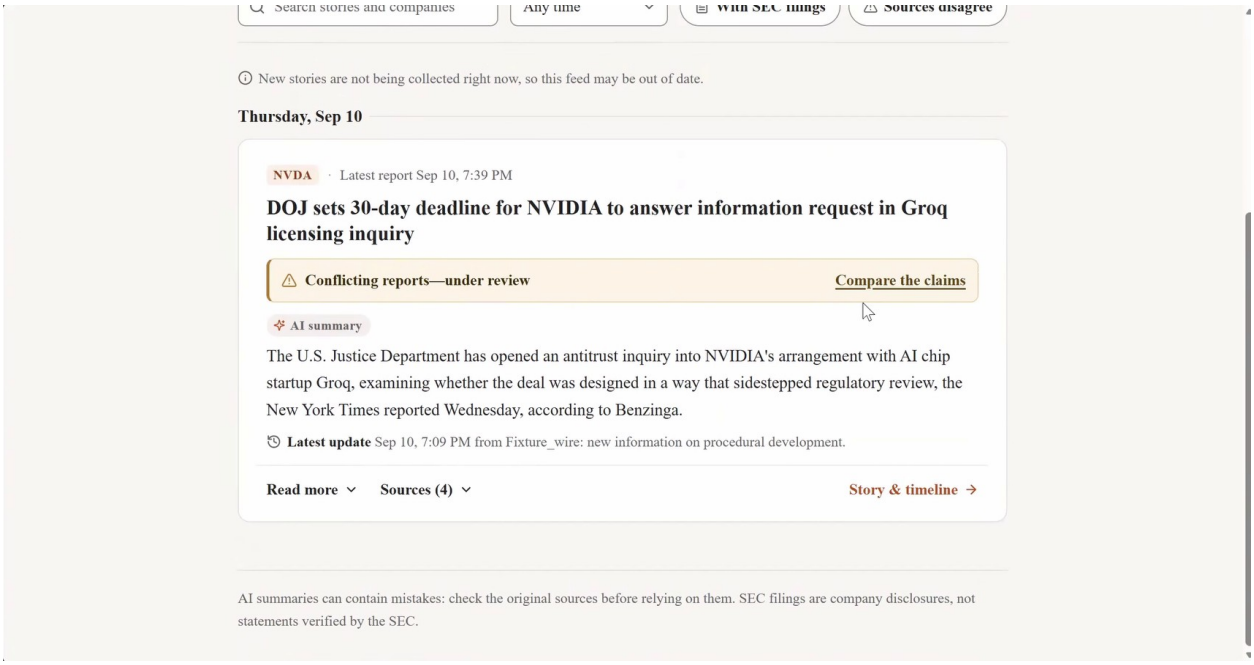


Figure A7. *The news feed marks the story "Conflicting reports — under review"*

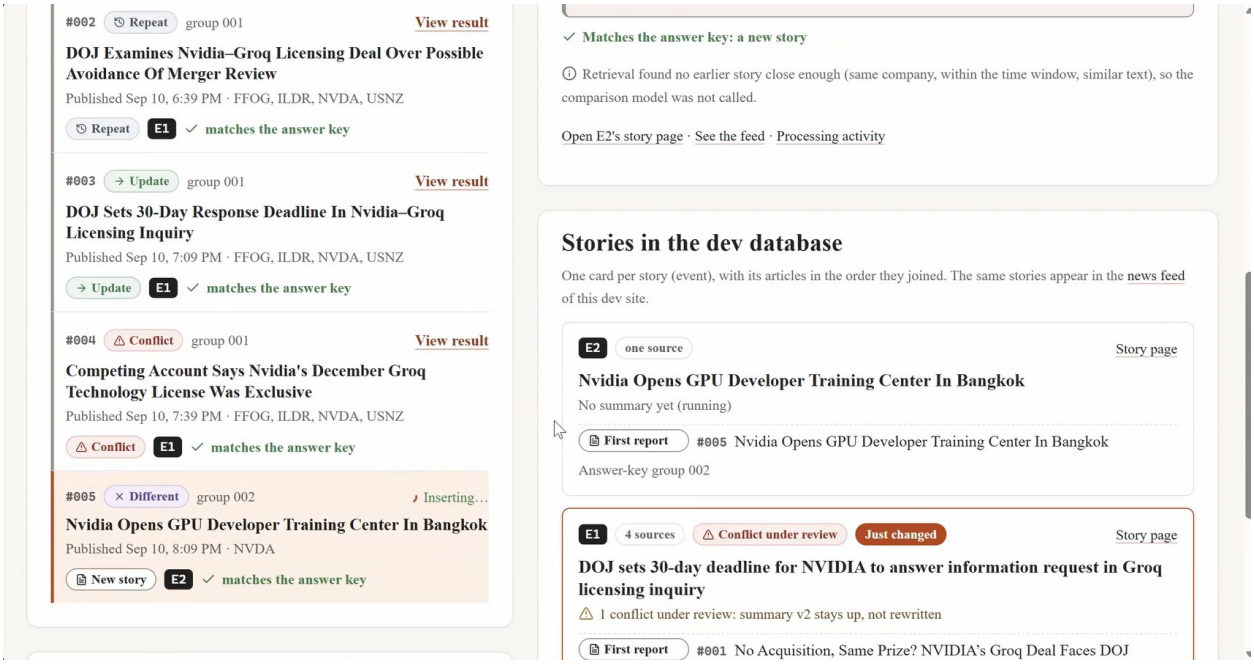


Figure A8. *A different NVIDIA event becomes its own story (E2)*

Appendix of **report**: Installation and User Guide

Set up

Run the following commands in **Windows PowerShell**, in order. The Docker setup requires Git and Docker Desktop. Start Docker Desktop before the database step. Have your Alpaca credentials ready for news collection and an Anthropic API key if you enable AI grouping or summaries.

1. Install `uv`, clone the repository, and install the project dependencies. If PowerShell does not recognise `uv` immediately after installation, reopen the terminal before continuing.

```
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"

git clone https://github.com/Qwenty228/GC1.git
cd GC1\SystemCode
uv sync --locked --extra local-embeddings
```

2. Create and edit `.env`. Do this before building the PostgreSQL image so the database and application use matching credentials. (copy content from .env.example and specify api key and postgres credentials)

```
Copy-Item .env.example .env
notepad .env
```

Variable	Value or action
`ALPACA_API_KEY`, `ALPACA_SECRET_KEY`	Enter your Alpaca credentials. Paper-trading keys can be used.
`SEC_USER_AGENT`	Enter the application name and a monitored contact email, for example `StockEventLens you@example.com`.
`INGEST_SYMBOLS`	`NVDA,MSFT,AAPL,GOOGL,AMZN,META,NFLX,DIS,JPM,V,GS`
`INGEST_LIVE_ONLY`	Use `true` for live collection. Use `false` when starting with recovery.
`ALPACA_RECOVERY_MAX_HOU`	Sets the maximum Alpaca lookback during recovery. The

`RS`	supplied configuration uses `3`.
`PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`	For the local Docker setup, use `localhost`, `5432`, `stock_news`, `postgres`, and your chosen password, respectively.
`SIMILARITY_ENABLED`, `SUMMARY_ENABLED`	Set both to `true` for the full grouping and summary workflow. Set both to `false` to avoid Anthropic model charges.
`ANTHROPIC_API_KEY`	Enter your key if either AI feature is enabled.
`SIMILARITY_COMPARISON_MODEL`, `SUMMARY_MODEL`	Use the configured model IDs and check that your Anthropic account has access to both.
`SUMMARY_STARTUP_CATCHUP_DAYS`	`3` enables summary catch-up for recent events at startup. `0` disables it.

3. Initialise the database and start the application.

If `make` is available on your system, run `make db`, otherwise run commands below:

```
docker compose up -d --wait postgres
uv run datasources-db migrate
uv run datasources-db populate
uv run datasources-db verify
```

Check that verification prints `Result: OK`, then continue.

```
uv run datasources-db register-companies
uv run datasources-similarity download-model
uv run datasources-dev --check
uv run datasources-dev
```

Open a web browser and navigate to http://localhost:8000 to access StockEvent Lens. Keep the PowerShell terminal running while using the application. To stop the server, press `Ctrl+C` in the terminal.

Daily collection and backfill

For new data, open PowerShell in `GC1\SystemCode` and run the following. With `INGEST_LIVE_ONLY=true`, collection begins at the startup cutoff. Use a second PowerShell window in the same directory for status checks.

```
uv run datasources-dev
```

To check	Command
Collection counts and source cursors	`uv run datasources-ingest status`
Grouping progress and events	`uv run datasources-similarity status`
Summary jobs and versions	`uv run datasources-summary status`
Events awaiting a summary	`uv run datasources-summary pending`

For a one-off backfill, stop the running collector with `Ctrl+C`, then run one capped collection pass. The command permits paid model calls and limits the run to 20 records and 60 model attempts.

```
uv run datasources-ingest bounded-run --source collect --max-records
20 --max-model-attempts 60 --confirm-paid-calls
```

To resume live collection and recover a longer gap in one start, use the following alternative. It raises the Alpaca recovery window from 3 hours to 24 hours for this terminal session.

```
$env:INGEST_LIVE_ONLY = "false"
$env:ALPACA_RECOVERY_MAX_HOURS = "24"
uv run datasources-dev
```

Alternatively, configure these values in the `.env` file to make the changes persistent.

Note: PowerShell environment variable overrides apply only to the current terminal session. To restore the default `.env` configuration, close the terminal and open a new PowerShell session before restarting the application.

Video recording of ingestion or data refresh

Live ingestion.mp4

Confirmed event with an SEC filing

The supplied database snapshot contains no corroborated event that includes an SEC filing. The three corroborated events each contain two news articles. The Microsoft filing is the closest example and can be reported as follows:

> StockEvent Lens ingested Microsoft's Form 8-K filed on 5 November 2026 (Item 5.02, accession `0001193125-26-448357`). The event remained provisional because the filing was its only linked source. During summary validation, the system flagged the figure "8" because it did not appear in the cited passage. The system withheld the summary and queued it for retry.

This example demonstrates **filing ingestion and citation validation**. To demonstrate **corroboration**, the proposed test is to process NVIDIA's 26 August earnings 8-K alongside the same-day news in a separate test database. Report it as a corroborated example only after verifying that the event has status `corroborated` and that its source list contains both the filing and an independent article. That test is still pending.

Appendix of **report**: Project Proposal

AI Tool Declaration

We used ChatGPT, Claude and Gemini to generate ideas, draft and refine paragraphs, improve expression, check the report's claims against our code and results, and create charts and illustrations. We are responsible for the content and quality of the submitted work.

References

- Bambrick, J., Xu, M., Almonte, A., Malioutov, I., Perarnau, G., Selo, V., & Chan, I. C. (2020). NSTM: Real-time query-driven news overview composition at Bloomberg. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations* (pp. 350–361). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-demos.40

- de Marneffe, M.-C., Rafferty, A. N., & Manning, C. D. (2008). Finding contradictions in text. In *Proceedings of ACL-08: HLT* (pp. 1039–1047). Association for Computational Linguistics. https://aclanthology.org/P08-1118/

- Escoter, L., Pivovarova, L., Du, M., Katinskaia, A., & Yangarber, R. (2017). Grouping business news stories based on salience of named entities. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers* (pp. 1096–1106). Association for Computational Linguistics. https://aclanthology.org/E17-1103/

- Fiscus, J., Doddington, G., Garofolo, J., & Martin, A. (1999). NIST's 1998 topic detection and tracking evaluation (TDT2). In *Proceedings of Eurospeech 1999* (pp. 247–250). https://doi.org/10.21437/Eurospeech.1999-65

- Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6465–6488). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.398

- Laban, P., Schnabel, T., Bennett, P. N., & Hearst, M. A. (2022). SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics, 10*, 163–177. https://doi.org/10.1162/tacl_a_00453

- Xu, L., Su, Z., Yu, M., Xu, J., Choi, J. D., Zhou, J., & Liu, F. (2024). Identifying factual inconsistencies in summaries: Grounding LLM inference via task taxonomy. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 14626–14641). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.857

- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36* (Datasets and Benchmarks Track). https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html